光照强度传感器

北京博创智联科技有限公司

2020-10-18

一、模块简介

光照强度传感器使用 I2C 接口进行通讯，模块 J1 接口连接到 UP 派扩展版的 P1 接口，J2 接口连接到平台的 P8 端口。

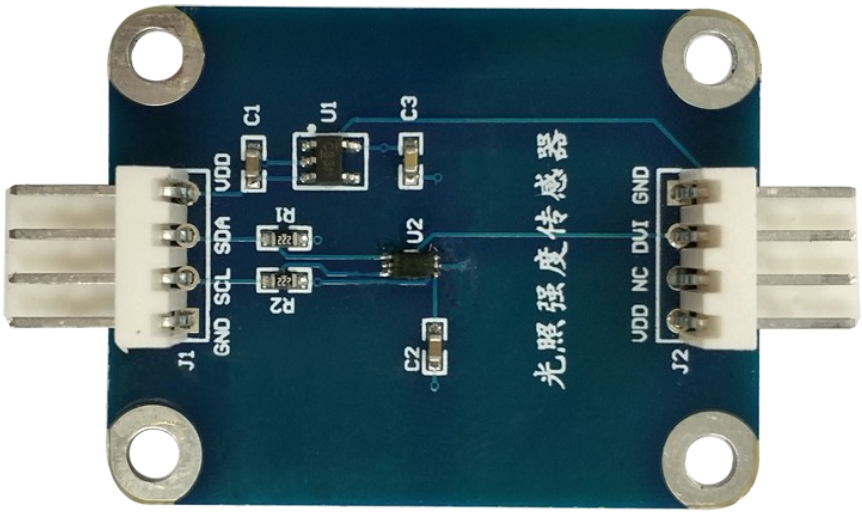

图 1.1 光照强度传感器

光照强度传感器主要由光敏元件组成。目前光敏元件发展迅速、品种繁多、应用广泛。市场出售的有光敏电阻、光敏二极管、光电三极管、硅光电池等。光照强度传感器可以感知周围光线情况，并告知处理芯片自动调节显示器背光亮度，降低产品的功耗。

二、操作步骤

1、线路连接

首先将光照强度传感器的 J1 接口通过连接线连接到 UP 派扩展板 P1 接口，J2 接口连接到的 P8 接口，然后使用串口线将 UP 派主板的串口 UART2 与电脑连接，最后连接 UP 派主板的电源线。

2、测试模块

打开超级终端或者其他串口工具，设置波特率为 115200，数据位为 8，奇偶校验为无，停止位为 1，数据流控制为无，设置完成后点击确定连接串口。打开平台的电源开关，待平台启动后，在串口终端中输入命令，进入模块目录：

```
root@imx8mmevk:~# cd /run/media/mmcblk1p1/modules/BH1750/
```

执行测试程序：

```
root@imx8mmevk:/run/media/mmcblk1p1/modules/BH1750# ./install.sh
```

3、运行结果

测试程序程序后，会打印出从传感器测量到的光照强度，单位为 lux 或 Lx，中文读作“勒克斯”。使用发光物理接近模块时，数值会发生改变。

终端运行效果：

```
BH1750:  67.50 lux
BH1750:  68.33 lux
BH1750:  68.33 lux
```

图 1.2 光照强度传感器运行效果

三、硬件原理

1、模块接口

光照强度传感器 J1 接口为一个 5V 电源引脚，一个 SCL 引脚，一个 SDA 引脚，一个 GND 引脚，对应 UP 派扩展板的 P1 接口。J2 接口为一个，5V 电源引脚，一个 DVI(GPIO)引脚，一个 GND 引脚，对应 UP 派扩展板的 P8 接口。

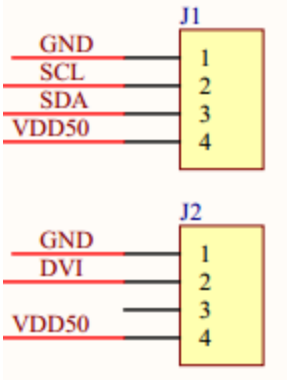

图 1.3 光照强度传感器接口原理图

2、光照强度传感器原理

本模块上主要是通过 BH1715 芯片获取数据。SDA 引脚为 I2C 数据引脚，SCL 引脚为 I2C 时钟引脚，DVI 引脚为 SDA，SCL 端口参考电压，DVI 端口为内部寄存器的异步重置端口。通过 I2C 接口进行通信即可控制芯片进行数据采集功能。

BH1715 指令简单，便于操作。只需要设置其通电状态和分辨率模式，即可使用连续读取方式读取数据。具体发送的指令请参考测试程序或芯片手册。

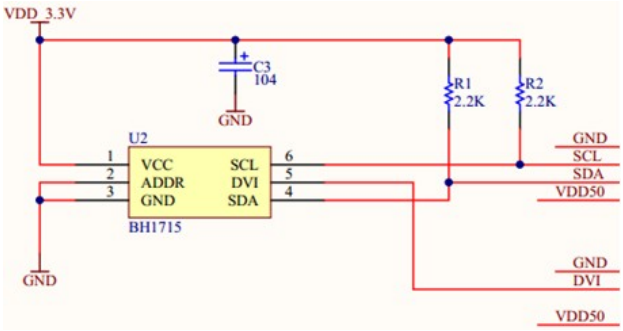

图 1.4 光照强度传感器原理图

四、Linux 下 I2C 驱动框架

I2C 驱动代码在内核中分布如下图所示：

```
uptech@uptech-virtual-machine:~/fsl-8mm-source/kernel-4.14.98$ ls drivers/i2c/
algos            i2c-core-base.c   i2c-core-smbus.c    i2c-smbus.c  muxes
busses           i2c-core.h        i2c-dev.c           i2c-stub.c
i2c-boardinfo.c  i2c-core-of.c     i2c-mux.c           Kconfig
i2c-core-acpi.c  i2c-core-slave.c  i2c-slave-eeprom.c  Makefile
```

图 4.1 I2C 驱动代码目录

I2c 的核心是 i2c-core.c 实现了 i2c 核心功能，i2c-dev.c 实现了与应用的交互接口。至于 algos 里面主要实现的是总线适配，busses 主要就是一些控制器相关的和 gpio 模拟的 i2c 代码。

Linux 的 I2C 体系结构符合 Linux 总线-设备-驱动开发模型，总体分为 3 个组成部分：

I2C 核心：将与具体适配器无关的代码以及探测设备，检测设备地址的上层代码提取到一块，为 I2C 总线驱动和设备驱动提供注册、注销以及通信方法，负责 I2C 驱动与 I2C 设备的匹配工作。

I2C 总线驱动：I2C 总线驱动是对 I2C 硬件体系结构中适配器端的实现，适配器可由 CPU 控制，甚至可以直接集成在 CPU 内部。

I2C 设备驱动：I2C 设备驱动(也称为客户驱动)是对 I2C 硬件体系结构中设备端的实现，设备一般挂接在受 CPU 控制的 I2C 适配器上，通过 I2C 适配器与 CPU 交换数据。Linux I2C 驱动架构图如下图所示：

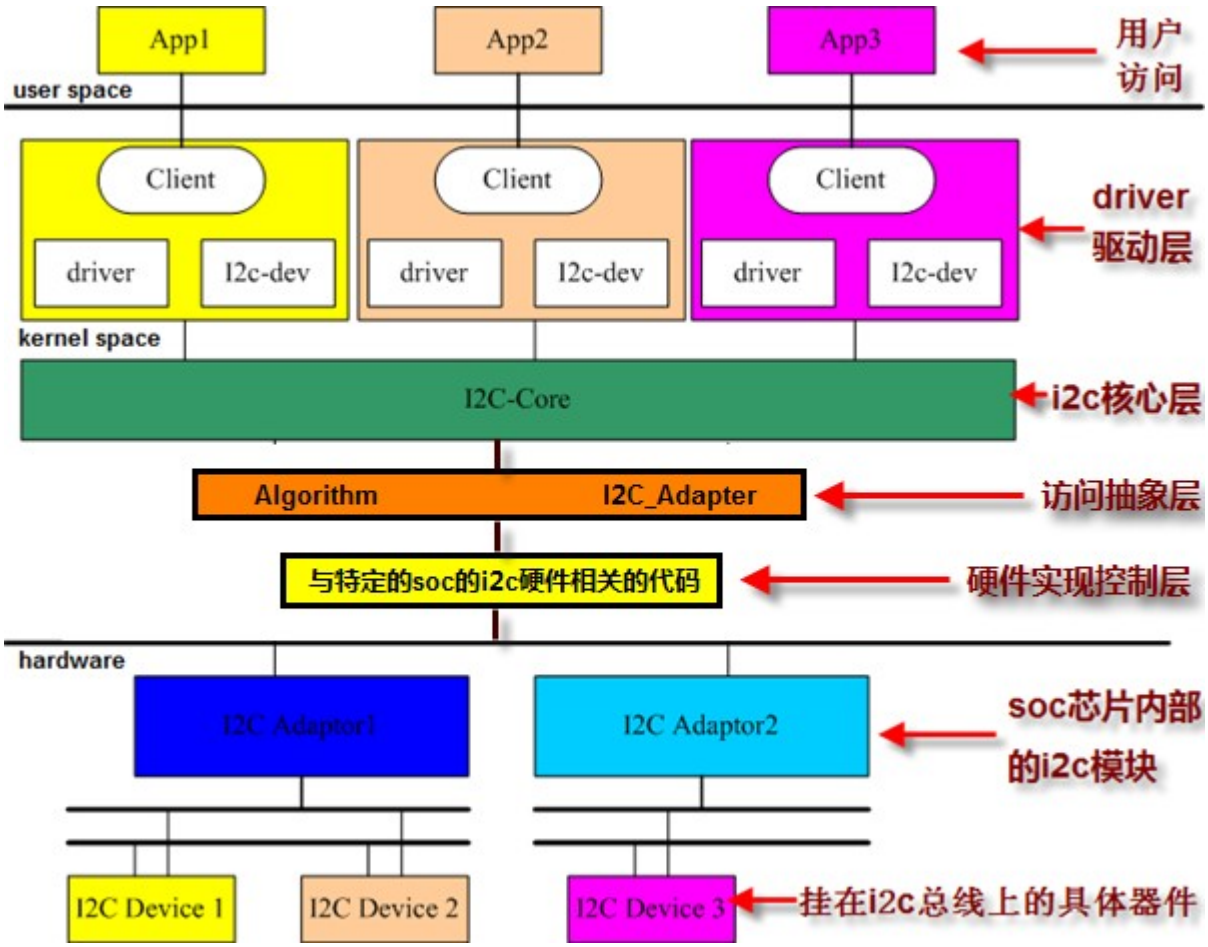

图 4.2 Linux i2c 驱动架构图

其中 adaptor 是 i2c 的总线驱动，驱动 CPU 内部的 i2c 适配器的工作，在内核引入设备树之后，这部分的工作变为修改设备树了，设备树内容下个章节会讲到。dev 和 driver，client 是 i2c 的设备驱动，提供用户层统一的 API，这部分的代码是通用的。

五、IMX8 I2C 设备树分析

分析 imx8 设备树文件 fsl-imx8mm-evk.dts，包括它包含的.dtsi 文件是"fsl-imx8mm.dtsi"和" fsl-imx8-ca53.dtsi"。提取 i2c 相关信息，可以发现 imx8 处理器上有四条 i2c 总线(在"fsl-imx8mm.dtsi"文件中)，i2c0 引用的是&i2c1 这个属性内容，“&”作用相当于 C 语言里面的指针。我们就以 i2c0 为例，带领读者分析一下 imx8mm 的 i2c 设备树，最终找到 i2c0 所使用 gpio 引脚。

```
i2c0 = &i2c1;
i2c1 = &i2c2;
i2c2 = &i2c3;
i2c3 = &i2c4;
```

继续查找 i2c1，在"fsl-imx8mm.dtsi"文件中有如下内容：

```
    i2c1: i2c@30a20000 {
```

```
        #address-cells = <1>;
        #size-cells = <0>;
        compatible = "fsl,imx8mm-i2c", "fsl,imx21-i2c";
        reg = <0x0 0x30a20000 0x0 0x10000>;
        interrupts = <GIC_SPI 35 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&clk IMX8MM_CLK_I2C1_ROOT>;
        status = "disabled";
    };
```

它定义了&i2c1 节点（也就是 i2c0）的各个属性及内容，具体属性含义可参考内核文档手册 Documentation/devicetree/bindings/i2c/下各个 txt 文档。

继续搜索&i2c1 节点，看在别的地方还有没有补充说明它的，发现在“fsl-imx8mm-evk.dts”中还有&i2c1 属性的一些内容，

```
&i2c1 {
    clock-frequency = <400000>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_i2c1>;
    status = "okay";
};
```

&i2c1 里面的子节点都是挂载在它上面的设备，pinctrl-0 = <&pinctrl_i2c1>;表明 pinctrl-0 属性的内容引用的是 pinctrl_i2c1。搜索 pinctrl_i2c1，得到内容如下：

```
pinctrl_i2c1: i2c1grp {
    fsl,pins = <
        MX8MM_IOMUXC_I2C1_SCL_I2C1_SCL          0x400001c3
        MX8MM_IOMUXC_I2C1_SDA_I2C1_SDA          0x400001c3
    >;
};
```

MX8MM_IOMUXC_I2C1_SCL_I2C1_SCL 和 MX8MM_IOMUXC_I2C1_SDA_I2C1_SDA 都是定义在“pins-imx8mm.h”中的宏定义，它们为五组数据，加上后面的 0x400001c3，组成六组数据供设备树解析代码处理，具体的解析过程需要详细阅读内核源代码。我们也可以通过名字看到，该宏定义的作用就是将 I2C1_SCL 配置为 i2c1 的 SCL，将 I2C1_SDA 配置为 i2c1 的 SDA。

我们打开 imx8mm 处理器的数据手册 IMX8MMRM.pdf，搜索 I2C1_SCL 和 I2C1_SDA 可以看到：

MUX_MODE	MUX Mode Select Field. Select 1 of 3 iomux modes to be used for pad: I2C1_SCL. 000 **ALT0** — Select signal I2C1_SCL 001 **ALT1** — Select signal ENET1_MDC 101 **ALT5** — Select signal GPIO5_IO14

1456 / 5367

图 5.1 I2C1_SCL 复用功能

MUX_MODE	MUX Mode Select Field. Select 1 of 3 iomux modes to be used for pad: I2C1_SDA. **NOTE:** Pad I2C1_SDA is involved in Daisy Chain. 000 **ALT0** — Select signal I2C1_SDA 001 **ALT1** — Select signal ENET1_MDIO - Configure register IOMUXC_ENET1_MDIO_SELECT_INPUT for mode ALT1. 101 **ALT5** — Select signal GPIO5_IO15

1457 / 5367 101.10%

图 5.2 I2C1_SDA 复用功能

I2C1_SCL 和 I2C1_SDA 引脚是可以配置成 i2c1 的 SDA 和 SCL，也可以配置成 GPIO5_IO14 和 GPIO5_IO15。

六、源码分析

1、测试程序

Bh1750_test.c 文件是光照强度传感器的数据采集源代码，该代码使用 Linux 系统调用函数 open、ioctl、read 和 write 操作 i2c 设备节点/dev/i2c-1，实现对传感器模块数据的读写。

源代码里面封装了两个函数 read_BH1750 和 write_BH1750 具体负责读写操作。

```
/*********************************************
 *函数名        : read_BH1750
 *描述          : 读取传感器数据
 *输入参数：
 *   @int fd          文件描述符
 *   @void *buff      缓存地址
 *   @size_t count        读取子节数
 *输出参数：*buff
 *返回值        : 出错:-1；   成功：返回读取的字节数
 *********************************************/
static int read_BH1750(int fd, void *buff, int count)
{
 int res;
 res=read(fd,buff,count);        //读取 count 个字节数据数据到 buff 缓存区
 return res;
}
/*********************************************
 *函数名        : write_BH1750
 *描述          : 向模块写入数据
 *输入参数：
 *   @int fd          文件描述符
 *   @unsigned char addr 写入地址
```

```
 *   @size_t count        写入子节数
 *返回值        ：出错:-1；   成功：写入的字节数
 ***********************************************/
static int write_BH1750(int fd, unsigned char addr, size_t count)
{
 int res;
 char  sendbuffer[count+1];
 sendbuffer[0] = addr;
 res=write(fd,sendbuffer,count);
}
```

在 main()函数中，首先使用 open（ ）函数打开 I2C_DEV

```
/*设备节点名字*/
#define I2C_DEV     "/dev/i2c-1"

fd = open(I2C_DEV, O_RDWR);         //打开 i2c 设备节点
 if(fd < 0){
     printf("####i2c test device open failed####\n");
     return (-1);
 }
```

然后使用 ioctl 进行操作配置，传感器模块设备地址位为 7bit，先使用 I2C_TENBIT 命令设置 i2c 设备地址位长度为 7bit(即参数为 0 ,参数为 1 则设置为 10bit) ,然后使用 I2C_SLAVE 命令设置从设备地址(这里定义宏 CHIP_ADDR 为从设备地址)

```
if(-1 == ioctl(fd,I2C_TENBIT,0)){       //设置地址位长度为 7bit
    printf("ioctl error on line %d\n",__LINE__);
    return (-1);
 }
 if(-1 == ioctl(fd,I2C_SLAVE,CHIP_ADDR)){   //设置从设备地址
    printf("ioctl error on line %d\n",__LINE__);
    return (-1);
 }
```

然后初始化 bh1750 模块，写入 0x01 命令，等待测量指令。

```
res = write_BH1750(fd, 0x01, 1);        //发送指令 0x01：等待测量指令
```

接下来发送 0x10 命令，设置为连续 H 分辨率模式

```
res = write_BH1750(fd, 0x10, 1);        //发送指令 0x10：连续 H 分辨率模式
```

最后，循环读取光照强度数据，并进行数值转换，打印最后结果：

```
while(1){
    usleep(120*1000);           //延时 120ms
    memset(buf,0,sizeof(buf));      //清零
    read_BH1750(fd,buf,2);          //读取光照强度数据
```

```
        flux = (float)(buf[0] << 8 | buf[1])/1.2;
        printf("BH1750: %6.2f lux\n", flux);   //打印输出
        fflush(stdout);
    }
```

这里使用 while(1)加 usleep( )方法，不停的读取数据，想要终止程序的话可以使用 CTRL+C 组合键强制退出该程序。

五、源码编译

1、驱动程序

先进入到驱动程序目录中，再执行 make 命令。

```
uptech@uptech-virtual-machine:~$ cd /imx8/modules/BH1750/driver/
uptech@uptech-virtual-machine:/imx8/modules/BH1750/driver$ make clean
uptech@uptech-virtual-machine:/imx8/modules/BH1750/driver$ LDFLAGS="" CC="$CC" make
```

编译成功后会在当前目录下生成 BH1750_driver.ko 文件。

2、测试程序

先进入到测试程序目录中，再执行 make 命令。

```
uptech@uptech-virtual-machine:~$ cd /imx8/modules/BH1750/test/
uptech@uptech-virtual-machine:/imx8/modules/BH1750/test$ source
/opt/fsl-imx-wayland/4.14-sumo/environment-setup-aarch64-poky-linux
uptech@uptech-virtual-machine:/imx8/modules/BH1750/test$ make clean
uptech@uptech-virtual-machine:/imx8/modules/BH1750/test$ make
```

编译成功后会在当前目录下生成 BH1750_test 文件。

六、下载测试

1、启动 UP 派扩展板

将配套光照强度传感器插入 UP 派扩展板(模块 J1 接口接扩展板 P1 接口，模块 J2 接口接扩展板 P8 接口)。连好网线，串口线，5V 电源线，启动 UP 派套装平台，输入 root 进入系统，通过串口终端挂载宿主机实验目录。

设置开发板 IP：192.168.12.199 (默认宿主机 ubuntu IP 192.168.12.157，NFS 共享目录/imx8)。

```
root@imx8mmevk:~# ifconfig eth0 192.168.12.199
root@imx8mmevk:~# mount -t nfs 192.168.12.157:/imx8 /mnt/
```

2、进入串口终端的 NFS 共享实验目录

```
root@imx8mmevk:~# cd /mnt/modules/BH1750/
```

3、加载驱动程序

```
root@imx8mmevk:/mnt/modules/BH1750# insmod driver/BH1750_driver.ko
```

4、执行测试程序

执行测试程序：

```
root@imx8mmevk:/mnt/modules/BH1750# ./test/BH1750_test
```